FRONIUS SYMO GEN24 10 PLUS & RASPBERRY PI

PV Optimizer Documentation

Intelligent Real-Time Photovoltaic Self-Consumption Optimization & Dynamic Load Controller

Target HW: Raspberry Pi 5 (Bookworm)
Runtime: Python 3.10+ / FastAPI
Protocol: Fronius Solar API v1
Smart Plugs: TP-Link Kasa (Local UDP)

1. Executive Summary & Application Scope

PV Optimizer is a dedicated on-premises automation system engineered to maximize the self-consumption (*Eigenverbrauchsoptimierung*) of residential photovoltaic installations. Modern hybrid PV systems with battery storage frequently produce substantial power surpluses during peak sunlight hours. Because feed-in remuneration tariffs are significantly lower than grid purchase rates, consuming self-generated power locally yields substantial economic and ecological returns.

PV Optimizer continuously reads generation and consumption telemetry directly from a **Fronius Symo GEN24 10 Plus** hybrid inverter via its local REST API. It regulates high-wattage thermal storage (such as a 3.5 kW electric water immersion heater) via a **Raspberry Pi GPIO relay** alongside a fleet of distributed **TP-Link Kasa Wi-Fi smart plugs** powering electric heaters, dehumidifiers, fans, and appliances.

Zero Grid Import — PRIORITY 1

Eliminates costly grid draw by automatically shedding active consumers within 60 seconds whenever domestic power draw exceeds instantaneous solar and battery delivery.

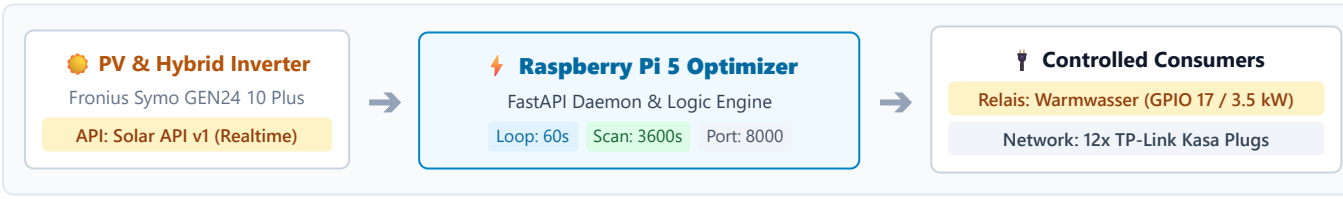
Battery Preservation — SOC PROTECTION

Enforces an adjustable battery state-of-charge threshold (e.g. 90%). All solar surplus is directed into home storage first before any thermal or auxiliary dump loads are triggered.

Thermal Energy Storage — 3.5 KW IMMERSION

Directs excess solar yield exceeding 3500 W into a domestic hot water cylinder via direct GPIO solid-state relay switching, effectively converting surplus electricity into heat.

2. High-Level System Architecture



3. Core System Specifications & Defaults

Parameter	Default Value	Operational Purpose
Inverter IP (`fronius_ip`)	`192.168.178.69`	Local IPv4 address of Fronius Symo GEN24 communication card (HTTP port 80).
Battery Threshold (`battery_threshold_percent`)	`90.0 %`	Minimum BYD battery SOC required before any surplus switching is permitted.
Warm Water Heater (`water_heater`)	`3500 W / GPIO 17`	High-priority thermal immersion dump load wired directly to Raspberry Pi Pin 11.
Optimization Cycle (`optimize_interval_sec`)	`60 seconds`	Periodic cadence for reading inverter telemetry and evaluating switching rules.
Network Scan Interval (`scan_interval_sec`)	`3600 seconds`	Hourly UDP discovery to detect new plugs or mark unplugged devices offline.

> **Autonomous Operation:** The system is fully self-contained. It requires no cloud subscriptions, no external broker (e.g. MQTT is not required), and operates seamlessly during Internet outages as all communication is strictly restricted to the local area network (LAN/WLAN) and physical GPIO pins.

4. Hardware Topology & Physical Interfacing

The PV Optimizer infrastructure bridges industrial photovoltaic hardware with standard domestic home automation appliances. The system coordinates three distinct physical interfaces: high-voltage solar and battery electronics, low-voltage embedded microcomputer GPIOs, and wireless smart home sockets.

Fronius Symo GEN24 10 Plus — SOLAR CORE

A three-phase hybrid inverter with integrated Multi Flow Technology, capable of managing DC PV input (up to 10 kWp) and bi-directional charging for a high-voltage BYD Battery-Box. Provides real-time telemetry through its onboard webserver via JSON/REST endpoints over local Wi-Fi or Ethernet.

Polled Endpoint:
`/solar_api/v1/GetPowerFlowRealtimeData.fcgi`

Raspberry Pi 5 (8GB / 4GB) — CONTROLLER NODE

Runs Debian-based Raspberry Pi OS (Bookworm 64-bit). Houses the Broadcom BCM2712 processor and custom RP1 I/O controller managing peripheral interfaces. Houses the FastAPI application daemon, background asyncio task loops, and physical GPIO switching circuitry.

Driver Library: `rpi-lgpio` (drop-in compatibility layer for RP1 silicon)

Relay & Water Immersion Heater — 3500 W THERMAL

A 3.5 kW electric heating element submerged in the domestic hot water buffer tank. Controlled via Raspberry Pi BCM Pin 17 driving an optocoupled relay board. The relay in turn drives a heavy-duty industrial AC contactor to switch the 230V/16A heating circuit safely.

Wiring: Pin 11 (GPIO 17) ➔ Optocoupler IN | Pin 6 ➔ GND

TP-Link Kasa Smart Plugs — 12X DISTRIBUTED

Wi-Fi connected smart plugs (KP115, HS110, HS100) distributed across the home. Connected to localized heating loads (kitchen, bathroom, study), air dehumidifiers, humidifiers, and ventilation fans. Communicates locally via port 9999 UDP/TCP autodiscovery and JSON control frames.

Discovery: Zero-configuration UDP Broadcast (port 9999)

5. Software Architecture & Component Stack

The software application is constructed in Python using an asynchronous, non-blocking architecture designed to ensure that network latency or temporary device dropouts never freeze the core optimization cycle.

FastAPI & Uvicorn
High-performance ASGI server handling concurrent REST API requests, websocket upgrades, and serving compiled static web dashboard assets.

python-kasa Engine
Asynchronous smart plug client performing automatic network discovery, status telemetry polling, and instantaneous remote actuation without cloud delays.

HTTPX Async Client
Non-blocking HTTP engine querying the Fronius Symo GEN24 Solar API. Implements short timeouts (10s) and graceful exception catching.

6. Electrical Circuit & Safety Isolation

> **Important Electrical Safety Requirement:** The Raspberry Pi GPIO pins operate at 3.3V DC logic level with maximum current sourcing capability of ~16 mA. Direct connection to an AC heating element will permanently destroy the Raspberry Pi and presents a severe electrical hazard.

Stage	Component Used	Safety & Operational Function
Logic Layer	Raspberry Pi GPIO Pin 17	3.3V DC digital logic output (active HIGH on trigger, LOW on idle).
Galvanic Isolation	Optocoupled Relay Module	Provides 2500V optical isolation between Raspberry Pi 5V rail and high-voltage circuit.
Power Switching	25A 2-Pole AC Contactor	Industrial contactor with integrated arc suppression safely switches 230V AC / 16A (3500 W).
Thermal Protection	Mechanical Temperature Cutoff	Dual safety thermostat on water tank cuts power independently at 65°C to prevent boiling.

7. Algorithmic Optimization Engine

The core intelligence of PV Optimizer resides within `pv_optimizer/logic.py`. The optimizer is triggered on a non-blocking 60-second timer loop. Every iteration executes a deterministic four-phase decision cycle designed to balance real-time energy flows with zero oscillation.

Phase 1: Ingestion

Queries Fronius Solar API:

- `P_PV` : Solar power (W)
- `P_Grid` : Grid net flow (W)
- `P_Load` : Domestic load (W)
- `SOC` : Battery charge (%)

Phase 2: Filtering

Filters manageable devices:

- `is_online == True`
- `is_enabled == True`
- `is_manual_override == False`
- Excludes disconnected plugs

Phase 3: Grid Draw Mitigation

If `P_Grid > 0` (Importing):

- Sort active plugs **descending**
- Turn OFF largest loads first
- Turn OFF Water Heater if needed
- Immediate grid neutral state

Phase 4: Surplus Allocation

If `P_Grid < 0` & `SOC ≥ 90%` :

- If surplus ≥ 3.5 kW: Warmwasser ON
- Sort inactive plugs **ascending**
- Turn ON fitting loads sequentially
- Maximize local absorption

8. Energy Flow Decision Matrix & Prioritization

The optimizer categorizes network states into three mutual operational regimes based on the sign of grid power (P_{Grid}) and battery SOC:

Condition	Grid State	Algorithm Action	Consumer Priority Sequence
Regime A: Grid Import `P_Grid > 0`	Household consumes power from public utility.	**Rapid Load Shedding:** Shuts down devices to eliminate purchase costs.	1. Plugs (Highest wattage ➔ Lowest) 2. Water Heater (3500 W)
Regime B: Battery Priority `P_Grid < 0` & `SOC < 90%`	Solar surplus exists, but battery storage is below reserve threshold.	**Inaction (Battery Protection):** Permits all surplus to flow into DC battery.	All auxiliary dump loads remain OFF. Battery takes 100% of available surplus.
Regime C: Surplus Dump `P_Grid < 0` & `SOC ≥ 90%`	Surplus exists AND home storage battery is satisfied.	**Cascaded Activation:** Turns ON loads that fit within calculated surplus.	1. Water Heater (if surplus ≥ 3.5 kW) 2. Plugs (Lowest wattage ➔ Highest)

9. State Management, Failsafes & Self-Healing

Offline Device Handling — SELF-HEALING

If a Kasa plug is unplugged, moved, or loses Wi-Fi connectivity, the background scanner detects the failure. The device is marked `is_online: false` and automatically set to `is_enabled: false` in `config.json`. This guarantees the optimizer will never attempt to activate an unreachable load.

Manual Override Bypass — USER PRIORITY

Any appliance can be flagged with `is_manual_override: true` through the Web Dashboard. The automated optimization algorithm immediately skips the consumer. The user can then toggle the device ON or OFF at will without the background loop reversing their decision.

> **Oscillation Damping & Inverter Hysteresis:** By evaluating real-time active power rather than estimated yield and sorting shed loads from largest to smallest, the optimizer prevents "hunting" (rapid cyclic toggling). A load is only activated if instantaneous surplus strictly exceeds its rated nominal wattage.

10. Web Dashboard & Interactive Interface

PV Optimizer serves an integrated single-page application directly from its FastAPI backend. The interface is accessible from any browser on the local network (PC, tablet, or smartphone) at `http://<raspberry-ip>:8000`.

Live Telemetry Dashboard (`index.html`)

UI OVERVIEW

- **4 Key Metrics Tiles:** Real-time display of PV generation (W), domestic load (W), grid import/export badge (W), and battery SOC (%).
- **Consumer Cards:** Displays power draw, online status indicator, manual override switch, and direct toggle button.
- **Instant Scan Button:** Triggers an ad-hoc network scan for smart plugs with live UI status notifications.

Hardware Simulation Console (`test.html`)

DEV & TESTING

- **Mock Inverter Sliders:** Manually vary simulated PV generation (0-10 kW), base house consumption, and battery percentage.
- **Mock GPIO Relays:** Emulates physical pin switching in memory for safe, hardware-free development on PC/Mac.
- **Algorithm Verification:** Inspect switching behavior under synthetic edge cases (e.g. cloud passage, battery depletion).

11. Complete REST API Specification

Method	Endpoint	Payload / Params	Description & Behavior
GET	`/api/status`	*None*	Returns current PV generation, grid power, load power, and battery SOC from the inverter.
GET	`/api/consumers`	*None*	Returns list of all consumers with state, nominal power, override flag, and online status.
POST	`/api/consumers/{id}/override`	`{"is_manual_override": bool}`	Enables or disables manual override on a specific consumer (exempting it from automation).
POST	`/api/consumers/{id}/toggle`	`{"is_on": bool}`	Actuates a consumer ON or OFF. Returns 400 error if manual override is not enabled.
GET	`/api/config`	*None*	Returns complete system configuration including plug lists and simulation flags.
POST	`/api/config`	`ConfigUpdate` model	Persists updated parameters to `config.json` and reinitializes active objects.
POST	`/api/scan`	*None*	Executes an immediate Kasa discovery scan and updates configuration automatically.
POST	`/api/mock`	`{"pv_power", "load", "soc"}`	Updates simulated values when application is started in `--mock-inverter` mode.

12. Declarative Configuration (`config.json`)

The application persists all settings in a human-readable `config.json` file located in the root working directory. Whenever newly discovered plugs are identified or missing plugs are marked offline, the file is automatically updated via callback.

> **Safe Hot-Reload:** Saving parameters through the Web UI updates both disk storage and runtime memory without requiring a service restart.

```
{
  "fronius_ip": "192.168.178.69",
  "battery_threshold_percent": 90.0,
  "scan_interval_sec": 3600,
  "optimize_interval_sec": 60,
  "water_heater": {
    "name": "Warmwasser",
    "power_w": 3500,
    "gpio_pin": 17
  },
  "plugs": [
    {
      "name": "Switch_2_Heizung_Kueche",
      "ip": "192.168.178.102",
      "power_w": 2000.0,
      "is_enabled": true
    }
  ]
}
```

13. Deployment on Raspberry Pi 5 (Debian Bookworm)

Raspberry Pi OS (Bookworm) enforces strict package isolation (PEP 668) prohibiting unmanaged `pip` installations into the global system Python environment. Furthermore, the Raspberry Pi 5 introduces the **RP1 peripheral controller chip**, replacing legacy memory-mapped GPIO access with character device drivers. Follow this standardized production setup:

Step 1: System Preparation & Tooling

OS SETUP

Update packages and install core build tools:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3-venv python3-pip git
```

Step 2: Repository Deployment

FILESYSTEM

Clone application files to the user home directory:

```
cd ~
git clone <repo-url> PVOptimizer
cd PVOptimizer
```

Step 3: Isolated Python Virtual Environment

PEP 668 COMPLIANT

Initialize virtualenv and install dependencies + RP1 driver:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install rpi-lgpio
```

Step 4: Manual Verification Run

SMOKE TEST

Start daemon interactively to confirm network reachability:

```
python main.py --host 0.0.0.0 --port 8000
# Open http://<Pi-IP>:8000 in browser
# Stop test with Ctrl + C
```

14. Production Autostart via Systemd (`pvoptimizer.service`)

To ensure uninterrupted 24/7 background operation across power outages or reboots, register the application as a system-managed daemon service at `/etc/systemd/system/pvoptimizer.service`:

```
[Unit]
Description=PV Optimizer Web Dashboard & Logic Engine
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/PVOptimizer
Environment="PATH=/home/pi/PVOptimizer/venv/bin:$PATH"
ExecStart=/home/pi/PVOptimizer/venv/bin/python main.py --host 0.0.0.0 --port 8000
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Service Registration Commands

CLI CON

Reload systemctl daemon, enable auto launch:

```
sudo systemctl daemon-reload
sudo systemctl enable pvoptimizer.
sudo systemctl start pvoptimizer.s
```

Diagnostic & Status Inspection

OPE

Verify active running state and tail applicat

```
sudo systemctl status pvoptimizer.
sudo journalctl -u pvoptimizer.ser
```

15. Operational Troubleshooting & Health Checks

Symptom	Probable Root Cause	Recommended Corrective Action
Inverter status shows 0 W	Fronius IP changed or web server disabled.	Set static DHCP lease in router; verify port 80 accessibility via `curl http://<IP>/solar_api/v1/...`
Relay clicking not occurring	Missing `rpi-lgpio` package on Pi 5.	Ensure `pip install rpi-lgpio` is installed inside the virtualenv (not legacy `RPi.GPIO`).
Plugs flagged as offline	Smart plug Wi-Fi disconnected or subnet blocked.	Verify plug LED is green; click "Suchen (Scan)" in UI to refresh UDP discovery table.